Simuler une application TinyOS dans Cooja

Cette procédure vaut pour n'importe quel programme TinyOS : un exemple fourni (Blink, RadioCountToLeds…), un exercice de TP ou votre propre projet. Elle suppose un PC sous Ubuntu où TinyOS, nesC, le compilateur MSP430, Java et Contiki-NG (qui contient Cooja) sont installés.

Vue d'ensemble

```
1. Écrire ou choisir l'application   →  fichiers .nc + Makefile
2. Compiler pour une carte simulable  →  make telosb  →  build/telosb/main.exe
3. Ouvrir Cooja                       →  ./gradlew run
4. Créer une simulation               →  File → New simulation
5. Créer un type de carte             →  Sky mote + main.exe
6. Ajouter et placer les cartes       →  positions, portée radio
7. Ouvrir les fenêtres d'observation  →  Mote output, Radio messages, Timeline
8. Lancer, observer, enregistrer      →  Start, puis File → Save simulation
```

> Règle à retenir : Cooja exécute le **binaire compilé** (`main.exe`), pas le code source. Après chaque modification du code, il faut recompiler (étape 2), puis recharger le binaire dans Cooja (étape 8, « Reload »).

Étape 0 · Vérifier l'environnement

```
nescc --version          # compilateur nesC
msp430-gcc --version     # compilateur pour les cartes Sky/TelosB et Z1
java -version            # Java 17 ou plus récent
echo $TINYOS_ROOT_DIR    # doit afficher le dossier de TinyOS, par ex. /home/moi/tinyos-main
ls ~/contiki-ng/tools/cooja
```

Si `TINYOS_ROOT_DIR` est vide :

```
echo 'export TINYOS_ROOT_DIR=$HOME/tinyos-main' >> ~/.bashrc
source ~/.bashrc
```

Étape 1 · Préparer l'application

Cas A : un exemple fourni avec TinyOS

```
ls $TINYOS_ROOT_DIR/apps                       # Blink, RadioCountToLeds, Oscilloscope…
cp -r $TINYOS_ROOT_DIR/apps/RadioCountToLeds ~/tinyos-apps/
cd ~/tinyos-apps/RadioCountToLeds
```

On travaille sur une **copie**, pour ne pas abîmer les exemples d'origine.

Cas B : une nouvelle application

Une application TinyOS tient au minimum en 3 fichiers, dans un dossier sans espace ni accent dans son chemin :

FICHIER	RÔLE
`MonAppAppC.nc`	Configuration : déclare les composants et les relie avec `->`.
`MonAppC.nc`	Module : le comportement, écrit sous forme de réactions à des événements.
`Makefile`	Indique quel composant compiler (voir ci-dessous).

```
# Makefile minimal
COMPONENT=MonAppAppC
TINYOS_ROOT_DIR?=$(HOME)/tinyos-main
include $(TINYOS_ROOT_DIR)/Makefile.include
```

> Évitez les espaces et les accents dans le chemin du dossier (par exemple « Réseaux de capteurs ») : le système de compilation de TinyOS les gère mal. Travaillez dans `~/tinyos-apps/`.

Étape 2 · Compiler pour une carte que Cooja sait simuler

```
make telosb
```

Le binaire est produit dans `build/telosb/main.exe`. Vérifiez sa présence :

```
ls -l build/telosb/main.exe
```

COMMANDE	CARTE TINYOS	TYPE DE CARTE DANS COOJA
`make telosb`	TelosB (MSP430 + radio CC2420)	**Sky mote** (choix recommandé)
`make z1`	Zolertia Z1	Z1 mote

Les avertissements (`warning`) n'empêchent pas la compilation. Seules les lignes `error` bloquent.

Étape 3 · Ouvrir Cooja

```
cd ~/contiki-ng/tools/cooja
./gradlew run
```

Le premier lancement prend une à deux minutes (téléchargement des dépendances), puis environ 15 s les fois suivantes. Laissez ce terminal ouvert : le fermer ferme Cooja.

Étape 4 · Créer une simulation

1. Menu **File** → **New simulation...**
2. Donnez un nom à la simulation.
3. **Radio medium** : gardez **Unit Disk Graph Medium (UDGM): Distance Loss**. C'est le modèle le plus simple : un message est reçu si la distance est inférieure à la portée.
4. Cliquez sur **Create**.

Étape 5 · Créer un type de carte avec votre binaire

1. Menu **Motes** → **Add motes** → **Create new mote type** → **Sky mote...**
2. **Description** : un nom parlant, par exemple « Capteur » ou « Passerelle ».
3. **Browse**, puis sélectionnez `~/tinyos-apps/MonApp/build/telosb/main.exe`. Si la fenêtre ne propose que les fichiers `.sky`, choisissez le filtre « All files ».
4. Cliquez sur **Create**.

Pour que des cartes exécutent des programmes **différents** (par exemple une passerelle et des capteurs), répétez cette étape avec un second binaire. Chaque type de carte apparaît ensuite dans le menu **Motes** → **Add motes**.

Étape 6 · Ajouter et placer les cartes

1. Dans la fenêtre qui suit, indiquez le **nombre de cartes**.
2. **Positioning** : *Random* (aléatoire), *Linear* (en ligne) ou *Ellipse*.
3. Cliquez sur **Add motes**.
4. Dans la fenêtre **Network**, faites glisser une carte pour la déplacer. Pour une position exacte en mètres : clic droit sur la carte, puis *Mote tools*, puis *Mote Interface Viewer*, et dans la liste déroulante, *Position*.

> Numéros de carte : Cooja numérote les cartes 1, 2, 3… et écrit ce numéro dans la variable `TOS_NODE_ID` de chaque carte. Un même programme peut donc se comporter différemment selon la carte, par exemple `if (TOS_NODE_ID == 1)` pour la passerelle.

Régler la portée radio

Dans la fenêtre **Network**, menu **View**, cochez *Radio environment (UDGM)*. Cliquez sur une carte : le cercle vert montre la portée d'émission, le cercle gris la zone d'interférence. Pour modifier ces valeurs (par défaut 50 m et 100 m), passez par le menu **Tools** → **Radio medium…**, ou faites un clic droit dans la fenêtre Network selon votre version de Cooja.

Étape 7 · Ouvrir les fenêtres d'observation

Elles se trouvent dans le menu **Tools** :

FENÊTRE	CE QU'ELLE MONTRE
Network	Le terrain et les cartes. Le menu *View* permet d'afficher les numéros (*Mote IDs*), les LED, la portée radio et le trafic radio (*Radio traffic*).
Mote output	Le texte que chaque carte écrit sur son port série. Un champ de filtre permet de n'afficher qu'une carte.
Radio messages	Chaque paquet radio, avec l'émetteur, les récepteurs et le contenu en octets.
Timeline	L'activité dans le temps : radio allumée, émission, réception, LED. Pratique pour mesurer des délais.
Mote Interface Viewer	L'état d'une carte : LED, bouton, position, port série.

Étape 8 · Lancer, observer, enregistrer

- **Start** lance la simulation, **Pause** la met en pause, **Step** avance pas à pas.
- **Speed limit** règle la vitesse. Pour l'observation, gardez 100 % (temps réel). Sans limite, la simulation va aussi vite que le PC le permet.
- **Reload** (ou **File** → **Reload simulation**) recharge le binaire : c'est indispensable après chaque recompilation.
- **File** → **Save simulation as...** enregistre un fichier `.csc` : cartes, positions, binaires et fenêtres ouvertes. Pour le rouvrir plus tard, utilisez **File** → **Open simulation**, ou lancez directement :

```
cd ~/contiki-ng/tools/cooja
./gradlew run --args="/chemin/vers/ma_simulation.csc"
```

Afficher du texte depuis une application TinyOS

Par défaut, la plupart des exemples TinyOS n'écrivent rien : on ne voit que les LED. Pour lire des messages dans **Mote output**, il y a deux méthodes.

Méthode 1 (simple) : écrire du texte brut sur le port série

```
// Configuration (…AppC.nc)
components PlatformSerialC;
App.SerieControl -> PlatformSerialC;
App.UartByte     -> PlatformSerialC;

// Module (…C.nc)
uses interface StdControl as SerieControl;
uses interface UartByte;

void ecrire(const char *s) { while (*s) call UartByte.send(*s++); }

event void Boot.booted() {
  call SerieControl.start();      // à faire une fois au démarrage
  ecrire("Bonjour\n");            // chaque \n = une ligne dans Mote output
}
```

Méthode 2 : la bibliothèque printf de TinyOS

Le composant `PrintfC` envoie des paquets série en binaire. Dans Mote output, ils apparaissent comme des caractères illisibles. Pour les lire, il faut ouvrir un *Serial Socket* sur la carte (clic droit, puis *Mote tools*, puis *Serial Socket (SERVER)*, qui indique un port, par exemple 60001), puis dans un autre terminal :

```
java net.tinyos.tools.PrintfClient -comm network@localhost:60001
```

Le même principe permet de brancher les outils Java de TinyOS (`Listen` , `Oscilloscope` ...) sur une carte `BaseStation` simulée.

Facultatif · Lancer sans fenêtre (tests automatiques)

Ajoutez un script de test à la simulation (menu **Tools** → **Simulation script editor**), enregistrez le `.csc`, puis :

```
cd ~/contiki-ng/tools/cooja
./gradlew run --args="--no-gui /chemin/vers/ma_simulation.csc"
```

Le script lit les lignes écrites par les cartes (variables `id` et `msg`) et conclut par `log.testOK()` ou `log.testFailed()`. Le résultat est écrit dans `COOJA.testlog`.

```
TIMEOUT(60000);                          // abandon après 60 s simulées
while (true) {
  YIELD();                               // attend la prochaine ligne écrite
  log.log(time + " carte " + id + " : " + msg + "\n");
  if (msg.contains("Bonjour")) log.testOK();
}
```

En cas de problème

SYMPTÔME	CAUSE PROBABLE ET SOLUTION
`Makefile.include` introuvable	`TINYOS_ROOT_DIR` n'est pas défini : voir l'étape 0, puis ouvrir un nouveau terminal.
Erreur de compilation sur un chemin coupé en deux	Espace ou accent dans le chemin du dossier : déplacer l'application dans `~/tinyos-apps/`.
Les modifications du code ne changent rien dans Cooja	Le binaire n'a pas été recompilé (`make telosb`) ou pas rechargé (**Reload**).
Mote output reste vide	L'application n'écrit rien sur le port série, c'est normal pour Blink. Voir « Afficher du texte ».
Caractères illisibles dans Mote output	L'application utilise `PrintfC` : voir la méthode 2.
Aucun message reçu entre deux cartes	Distance supérieure à la portée radio (voir l'étape 6), ou numéro de destination incorrect dans `AMSend.send(...)`.
`./gradlew run` reste bloqué sans ouvrir de fenêtre	Un autre Cooja tourne déjà dans ce dossier : le fermer avant d'en relancer un.